

Regression Analysis

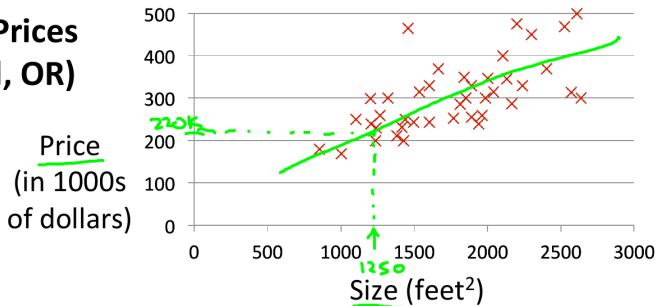
Assit.Prof.Dr.Emre Özbilge

Department of Artificial Intelligence Engineering
Faculty of Engineering
Cyprus International University

March 7, 2025

Linear Regression with One Variable

Housing Prices (Portland, OR)



- We provide the right answer to the learning system in order to obtain mathematical model that predicts the expected values when unseen input data is presented to the model. This is called supervised learning.
- Regression problem is when predicting real-valued output.

Data Description

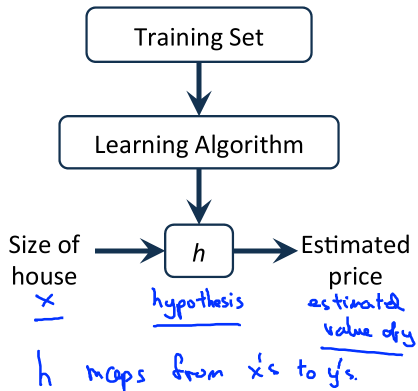
Training set of housing prices (Portland, OR)

Size in feet ² (x)	Price (\$) in 1000's (y)
→ 2104	460
1416	232
→ 1534	315
852	178
...	...

$m = 47$

- m = Number of training examples.
- x = inputs variable/features.
- y = output variable / target variable.
- $(x^{(i)}, y^{(i)})$ – i^{th} training example.
- e.g. $x^{(1)} = 2104$, $x^{(2)} = 1416$, $y^{(1)} = 460$

Model (Hypothesis) Representation



How do we represent h ?

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

Shorthand: $h(x)$



Linear regression with one variable. (x)
Univariate linear regression.
↳ one variable

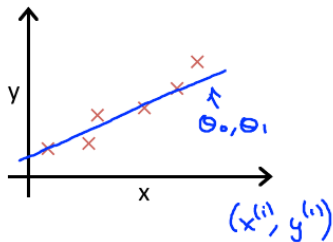
Cost Function I

Training Set	Size in feet ² (x)	Price (\$) in 1000's (y)
	2104	460
	1416	232
	1534	315
	852	178
	...	...

} $n = 47$

- Hypothesis: $h_{\theta}(x) = \theta_0 + \theta_1 x$
- θ_i – parameters

Cost Function II



Idea: Choose θ_0, θ_1 so that $\underline{h_\theta(x)}$ is close to $\underline{y}$ for our training examples $\underline{(x, y)}$

Hypothesis:

$$\underline{h_\theta(x) = \theta_0 + \theta_1 x}$$

Parameters:

$$\underline{\theta_0, \theta_1}$$



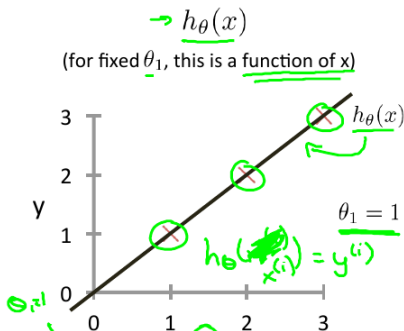
Cost Function:

$$\rightarrow J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2$$

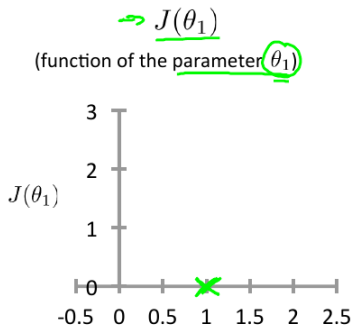
Goal: minimize $J(\theta_0, \theta_1)$
 $\nearrow_{\theta_0, \theta_1}$

Cost Function III

- Assume θ_0 is removed in order to simplify the problem.
- $\theta_1 = 1$



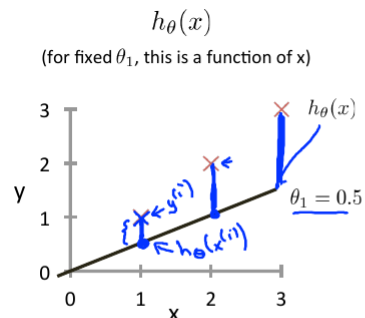
$$\begin{aligned} J(\theta_1) &= \frac{1}{2m} \sum_{i=1}^m (h_\theta(x^{(i)}) - y^{(i)})^2 \\ &= \frac{1}{2m} \sum_{i=1}^m (\theta_1 x^{(i)} - y^{(i)})^2 = \frac{1}{2m} (0^2 + 0^2 + 0^2) = 0^2 \end{aligned}$$



$$\begin{aligned} \theta_1 &= 0.5? \\ J(1) &= 0 \end{aligned}$$

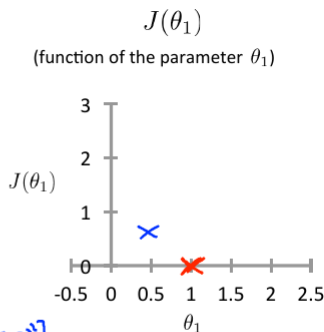
Cost Function IV

- $\theta_1 = 0.5$



$$J(0.5) = \frac{1}{2m} [(0.5-1)^2 + (1-2)^2 + (1-3)^2]$$

$$= \frac{1}{2 \times 3} (3.5) = \frac{3.5}{6} \approx \underline{0.58}$$

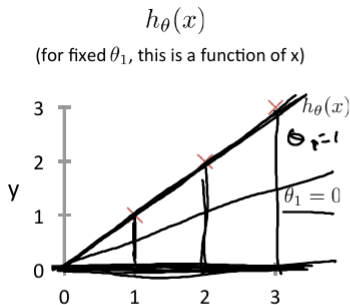


$$\theta_1 = 0?$$

$$J(0) = ?$$

Cost Function V

- $\theta_1 = 0$

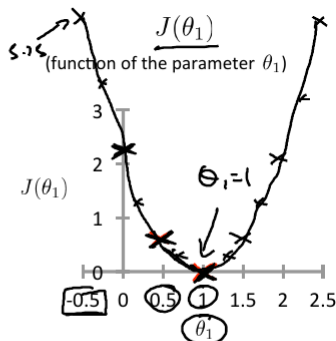


$$J(0) = \frac{1}{2m} (1^2 + 2^2 + 3^2)$$

$$= \frac{1}{6} \cdot 14 \approx 2.3$$

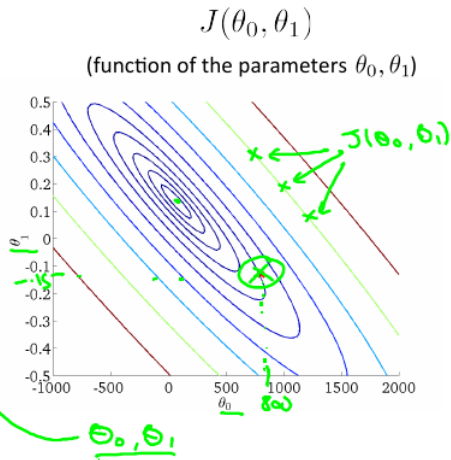
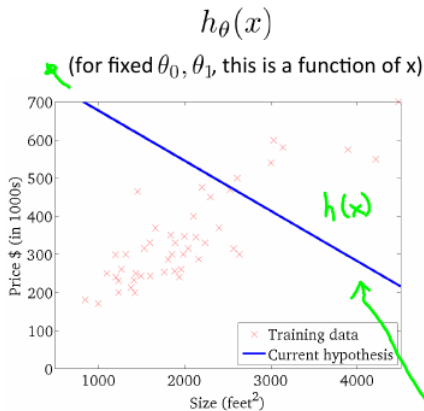
$$h(x) = -0.5x$$

minimize $J(\theta_1)$
 θ_1



Cost Function VI

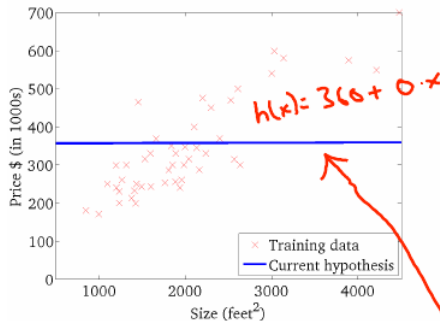
- Illustrating the learning algorithm for both model's parameters.



Cost Function VII

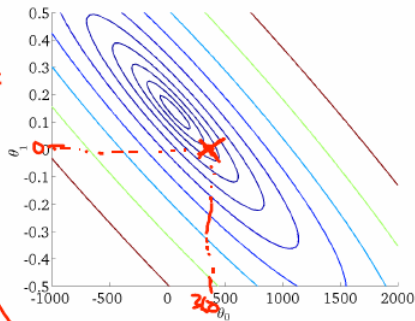
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

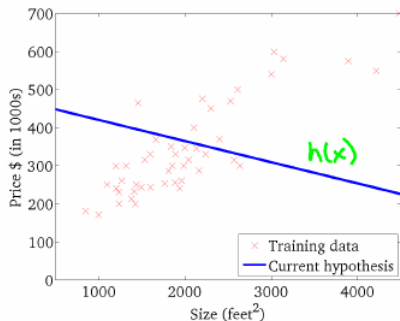
(function of the parameters θ_0, θ_1)



Cost Function VIII

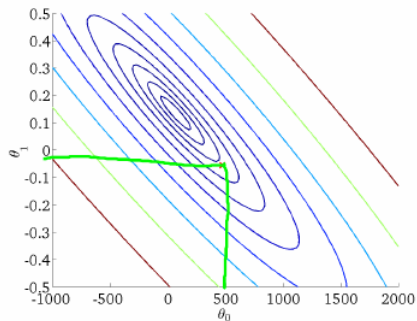
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

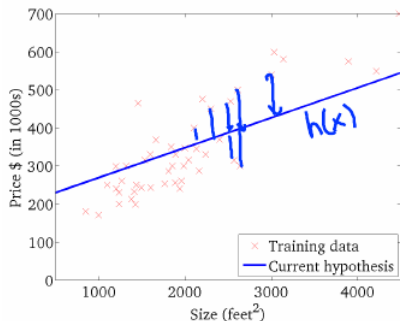
(function of the parameters θ_0, θ_1)



Cost Function IX

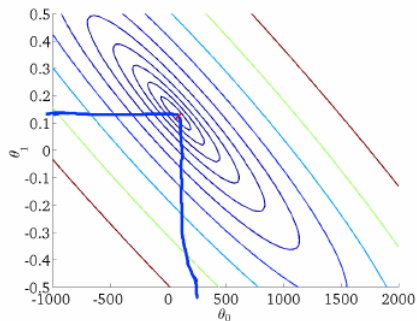
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

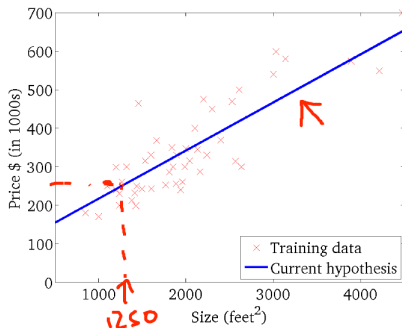
(function of the parameters θ_0, θ_1)



Cost Function X

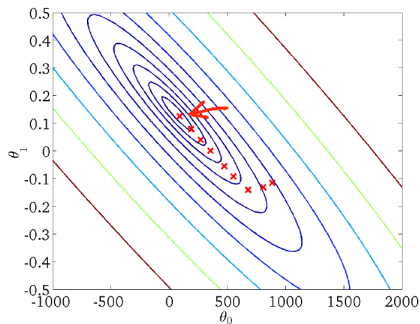
$$h_{\theta}(x)$$

(for fixed θ_0, θ_1 , this is a function of x)



$$J(\theta_0, \theta_1)$$

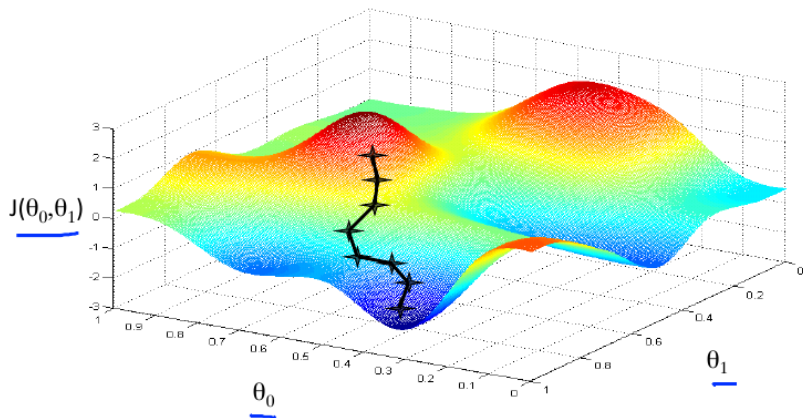
(function of the parameters θ_0, θ_1)



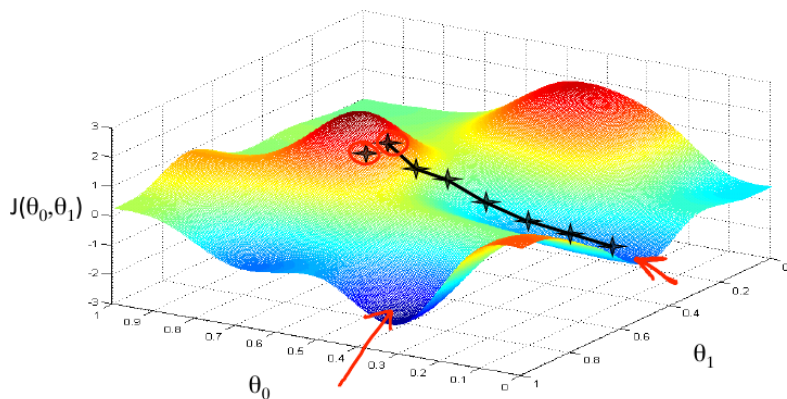
Gradient Descent Algorithm I

- have some function $J(\theta_0, \theta_1)$ (here it can be multi-inputs system as well $J(\theta_0, \theta_1, \theta_2, \dots, \theta_n)$)
- want $\arg \min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$ (or $\arg \min_{\theta_0, \dots, \theta_n} J(\theta_0, \dots, \theta_n)$)
- start with some θ_0, θ_1
- keep changing θ_0, θ_1 to reduce $J(\theta_0, \theta_1)$ until we hopefully end up at a minimum.

Gradient Descent Algorithm II



Gradient Descent Algorithm III



Gradient Descent Algorithm IV

Gradient descent algorithm

repeat until convergence {
 $\theta_j := \theta_j - \alpha \frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$
 (for $j = 1$ and $j = 0$)
}

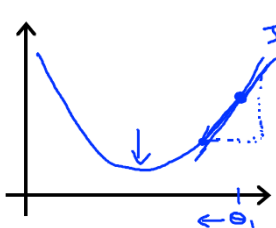
Linear Regression Model

$$h_{\theta}(x) = \theta_0 + \theta_1 x$$

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2$$

- α is learning rate and $\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1)$ is derivative of the cost function respect with θ_j .

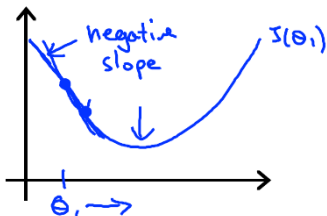
Gradient Descent Algorithm V



$$J(\theta_1) \quad (\theta_1 \in \mathbb{R})$$

$$\theta_1 := \theta_1 - \underbrace{\left(\frac{\partial}{\partial \theta_1} J(\theta_1) \right)}_{\geq 0} \cdot \underbrace{\alpha}_{\leftarrow}$$

$$\theta_1 := \theta_1 - \alpha \cdot (\text{positive number})$$



$$\frac{\partial}{\partial \theta_1} J(\theta_1) \leq 0$$

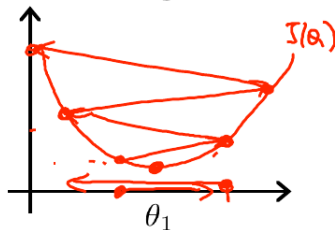
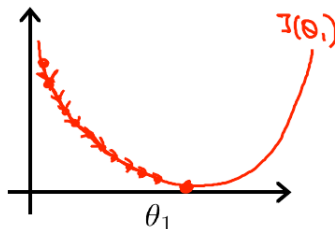
$$\theta_1 := \theta_1 - \underbrace{\alpha}_{\uparrow} \cdot \underbrace{(\text{negative number})}_{\uparrow}$$

Gradient Descent Algorithm VI

$$\theta_1 := \theta_1 - \alpha \frac{\partial}{\partial \theta_1} J(\theta_1)$$

If α is too small, gradient descent can be slow.

If α is too large, gradient descent can overshoot the minimum. It may fail to converge, or even diverge.

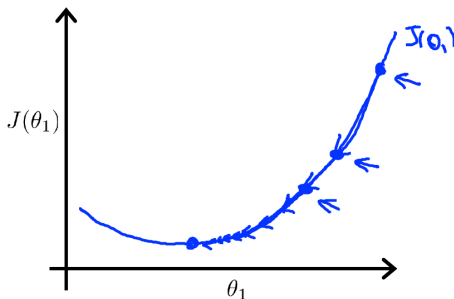


Gradient Descent Algorithm VII

Gradient descent can converge to a local minimum, even with the learning rate α fixed.

$$\theta_1 := \theta_1 - \alpha \frac{d}{d\theta_1} J(\theta_1)$$

As we approach a local minimum, gradient descent will automatically take smaller steps. So, no need to decrease α over time.



Gradient Descent Algorithm VIII

$$\begin{aligned}\frac{\partial}{\partial \theta_j} J(\theta_0, \theta_1) &= \frac{2}{2\theta_j} \frac{1}{2m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})^2 \\ &= \frac{2}{2\theta_j} \frac{1}{2m} \sum_{i=1}^m (\theta_0 + \theta_1 x^{(i)} - y^{(i)})^2\end{aligned}$$

$$j = 0 : \frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$j = 1 : \frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1) = \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

Gradient Descent Algorithm IX

- Lets we do some math. Chain rule is used to obtain the partial derivative of the cost function respect to the corresponding parameter.

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^m \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right)^2$$

$$\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_0} = \frac{1}{2m} \cdot 2 \sum_{i=1}^m \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right) \cdot 1$$

$$\frac{\partial J(\theta_0, \theta_1)}{\partial \theta_1} = \frac{1}{2m} \cdot 2 \sum_{i=1}^m \left(\theta_0 + \theta_1 x^{(i)} - y^{(i)} \right) \cdot x^{(i)}$$

Gradient Descent Algorithm X

Gradient descent algorithm

repeat until convergence {

$$\theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})$$

$$\theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) \cdot x^{(i)}$$

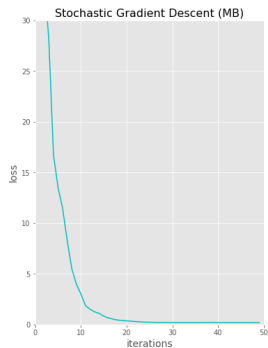
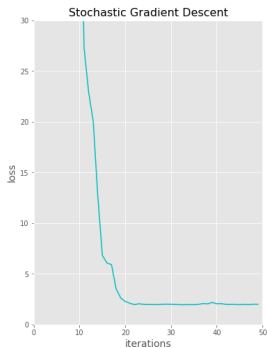
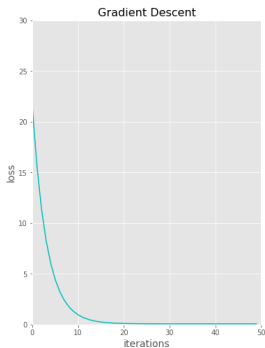
}

$$\frac{\partial}{\partial \theta_0} J(\theta_0, \theta_1)$$

update
 θ_0 and θ_1
simultaneously

$$\frac{\partial}{\partial \theta_1} J(\theta_0, \theta_1)$$

Gradient Descent Algorithm XI



Linear Regression with Multiple Variables

Multiple features (variables).

Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
x_1	x_2	x_3	x_4	y
2104	5	1	45	460
1416	3	2	40	232
1534	3	2	30	315
852	2	1	36	178
...	...	...	...	...

Notation:

- n = number of features $n=4$
- $x^{(i)}$ = input (features) of i^{th} training example.
- $x_j^{(i)}$ = value of feature j in i^{th} training example.

$$x^{(2)} = \begin{bmatrix} 1416 \\ 3 \\ 2 \\ 40 \end{bmatrix}$$
$$x_3^{(2)} = 2$$

Model (Hypothesis) Representation

$$\rightarrow h_{\theta}(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \dots + \theta_n x_n$$

For convenience of notation, define $x_0 = 1.$ ($x_0^{(i)} = 1$)

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \vdots \\ \theta_n \end{bmatrix} \in \mathbb{R}^{n+1}$$

$$h_{\theta}(x) = \theta_0 x_0 + \theta_1 x_1 + \dots + \theta_n x_n$$

$$= \boxed{\theta^T x}$$

θ^T is a $1 \times (n+1)$ matrix.
 x is a $(n+1) \times 1$ matrix.
 $\theta^T x$ is a scalar.

Multivariate linear regression. $\leftarrow$

Gradient descent for Multiple Variables

Gradient Descent

Previously ($n=1$):

Repeat {

$$\rightarrow \theta_0 := \theta_0 - \alpha \underbrace{\frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)})}_{\frac{\partial}{\partial \theta_0} J(\theta)}$$

$$\rightarrow \theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

(simultaneously update θ_0, θ_1)

}

New algorithm ($n \geq 1$):

Repeat {

$$\rightarrow \theta_j := \theta_j - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

(simultaneously update θ_j for $j = 0, \dots, n$)

}

$$\rightarrow \theta_0 := \theta_0 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_0^{(i)}$$

$$\rightarrow \theta_1 := \theta_1 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_1^{(i)}$$

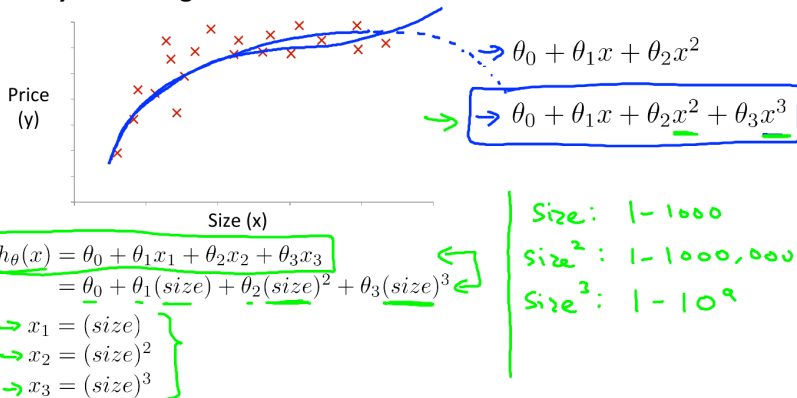
$$\rightarrow \theta_2 := \theta_2 - \alpha \frac{1}{m} \sum_{i=1}^m (h_{\theta}(x^{(i)}) - y^{(i)}) x_2^{(i)}$$

...

Polynomial Regression Analysis I

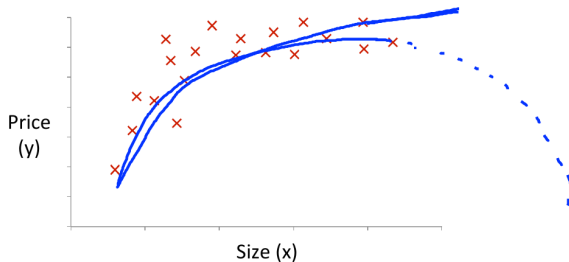
- When the data cannot be represented with linear models, simply we can generate nonlinear input features to obtain a non-linear model.

Polynomial regression



Polynomial Regression Analysis II

Choice of features



$$\rightarrow h_{\theta}(x) = \theta_0 + \theta_1(\text{size}) + \theta_2(\text{size})^2$$

$$\rightarrow h_{\theta}(x) = \theta_0 + \theta_1(\text{size}) + \theta_2\sqrt{(\text{size})}$$



Generate polynomial and interaction features:

- Lets we have three features: a, b, c
 1. `PolynomialFeatures(degree=3, interaction_only=True, include_bias=False)`
 - ▶ ['a', 'b', 'c', 'a b', 'a c', 'b c', 'a b c']
 2. `PolynomialFeatures(degree=3, interaction_only=False, include_bias=False)`
 - ▶ ['a', 'b', 'c', 'a²', 'a b', 'a c', 'b²', 'b c', 'c²', 'a³', 'a² b', 'a² c', 'a b²', 'a b c', 'a c²', 'b³', 'b² c', 'b c²', 'c³']

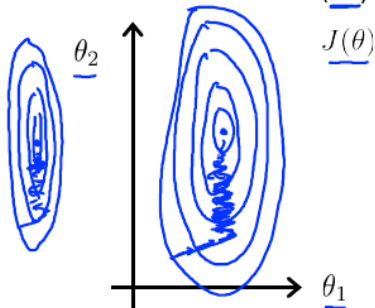
Feature Scaling I

Feature Scaling

Idea: Make sure features are on a similar scale.

E.g. $x_1 = \text{size (0-2000 feet}^2\text{)}$ ←

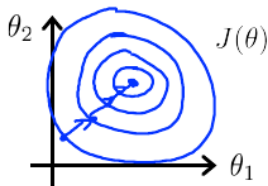
$x_2 = \text{number of bedrooms (1-5)}$ ←



$$\rightarrow x_1 = \frac{\text{size (feet}^2\text{)}}{2000} \quad \swarrow$$

$$\rightarrow x_2 = \frac{\text{number of bedrooms}}{5} \quad \searrow$$

$$0 \leq x_1 \leq 1 \quad 0 \leq x_2 \leq 1$$



Feature Scaling II

Mean normalization

Replace x_i with $x_i - \mu_i$ to make features have approximately zero mean
(Do not apply to $x_0 = 1$).

E.g. $\rightarrow x_1 = \frac{\text{size} - 1000}{2000}$

Average size = 1000

$x_2 = \frac{\# \text{bedrooms} - 2}{5 - 4}$

1-5 bedrooms

$\rightarrow -0.5 \leq x_1 \leq 0.5 \quad -0.5 \leq x_2 \leq 0.5$

$x_1 \leftarrow \frac{x_1 - \mu_1}{\sigma_1}$ ← avg. value of x_1 in training set

$x_2 \leftarrow \frac{x_2 - \mu_1}{\sigma_2}$

σ_1 range (max-min) (or standard deviation)

Feature Scaling III

- Rescaling (min-max normalization) to $[0, 1]$:

$$x' = \frac{x - \min(x)}{\max(x) - \min(x)} \quad (1)$$

- Rescaling (min-max normalization) to $[a, b]$:

$$x' = a + \frac{(x - \min(x)) \cdot (b - a)}{\max(x) - \min(x)} \quad (2)$$

- Mean normalization:

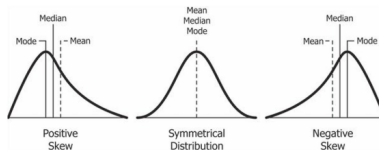
$$x' = \frac{x - \text{mean}(x)}{\max(x) - \min(x)} \quad (3)$$

- Standardization (Z-score Normalization):

$$x' = \frac{x - \text{mean}(x)}{\text{stdev}(x)} \quad (4)$$

Feature Scaling IV

- Logarithmic Transformations:
 - Reducing the Impact of Outliers



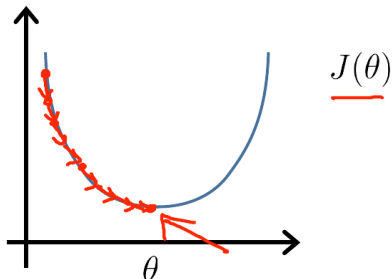
- Transforming Skewed Data:
- Linearizing Relationships Between Variables
- Stabilizing Variance in Heteroscedastic (having different variances) Data

$$x_{\log} = \log_e(x_{\text{raw}} + C) \quad (5)$$

$$x_{\log} = \log_{10}(x_{\text{raw}} + C) \quad (6)$$

Normal Equation Method for Regression Analysis I

Gradient Descent



Normal equation: Method to solve for θ analytically.

Normal Equation Method for Regression Analysis II

Examples: $m = 4$.

	Size (feet ²)	Number of bedrooms	Number of floors	Age of home (years)	Price (\$1000)
x_0	x_1	x_2	x_3	x_4	y
1	2104	5	1	45	460
1	1416	3	2	40	232
1	1534	3	2	30	315
1	852	2	1	36	178

$X = \begin{bmatrix} 1 & 2104 & 5 & 1 & 45 \\ 1 & 1416 & 3 & 2 & 40 \\ 1 & 1534 & 3 & 2 & 30 \\ 1 & 852 & 2 & 1 & 36 \end{bmatrix}$
 $m \times (n+1)$

$y = \begin{bmatrix} 460 \\ 232 \\ 315 \\ 178 \end{bmatrix}$
 m -dimensional vector
 $m \times 1$

$\theta = (X^T X)^{-1} X^T y$

Normal Equation Method for Regression Analysis III

- Lets we do some linear algebra calculation to prove the equation of parameter θ .
- We have a system as follows:

$$\begin{aligned}\mathbf{X}\theta &= \mathbf{y} & \mathbf{X} &\in m \times (n+1) \\ & & \mathbf{y} &\in m \times 1 \\ & & \theta &\in (n+1) \times 1\end{aligned}$$

- The goal is to obtain the parameter θ to produce the best fit solution to $\mathbf{X}\theta \approx \mathbf{y}$.
- So we will try to minimise the error $\epsilon = \mathbf{y} - \mathbf{X}\theta$.
- We take the square of the error function, so the objective becomes:

$$J(\theta) = \|\epsilon\|^2 = \|\mathbf{y} - \mathbf{X}\theta\|^2$$

Normal Equation Method for Regression Analysis IV

- Minimisation:

- $J(\theta) = \arg \min_{\theta} ||y - X\theta||^2$
- $J(\theta) = ||y - X\theta||^2 = (y - X\theta)^T (y - X\theta) =$
- $y^T y - y^T X\theta - (X\theta)^T y + (X\theta)^T (X\theta) =$
- $y^T y - y^T X\theta - \theta^T X^T y + \theta^T X^T X\theta =$
- $y^T y - 2\theta^T X^T y + \theta^T X^T X\theta = \quad \theta^T X^T X\theta = X^T X\theta^2$
- $\frac{\partial J(\theta)}{\partial \theta} = -2X^T y + 2X^T X\theta = 0$
- $2X^T X\theta = 2X^T y \rightarrow X^T X\theta = X^T y$
- To find θ , we have to multiply both side with inverse of $X^T X$
 $(X^T X)^{-1}(X^T X)\theta = (X^T X)^{-1}X^T y$
 $(X^T X)^{-1}(X^T X)$ is identity matrix I
- $\theta = (X^T X)^{-1}X^T y = X^\dagger y$
where $X^\dagger = (X^T X)^{-1}X^T$ is the pseudoinverse of X

$$(AB)^T = B^T A^T, \quad AA^{-1} = I, \quad IA = AI = A \quad (I \text{ is identity matrix})$$

Normal Equation Method for Regression Analysis V

$$\theta = \boxed{(X^T X)^{-1} X^T y} \leftarrow$$

$(X^T X)^{-1}$ is inverse of matrix $X^T X$.

Set $A = \underline{X^T X}$

$$\boxed{(X^T X)^{-1}} = A^{-1}$$

Octave: `pinv(X' * X) * X' * y`

$$\underline{\text{pinv}(X^T * X) * X^T * y}$$

$$\theta = (X^T X)^{-1} X^T y$$

$\min J(\theta)$

$$\begin{array}{c|c} X' & X^T \\ \hline \text{Feature Scaling} & \\ 0 \leq x_1 \leq 1 & \\ 0 \leq x_2 \leq 1000 & \\ 0 \leq x_3 \leq 10^{-5} & \checkmark \end{array}$$

Normal Equation Method for Regression Analysis VI

- **Linear algebra:** A Method for Inverting Matrices.
- To find the inverse of an invertible matrix A , find a sequence of elementary row operations that reduces A to the identity and then perform that same sequence of operations on I_n to obtain A^{-1} .
- **Example 1:**

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 5 & 3 \\ 1 & 0 & 8 \end{bmatrix}$$

Normal Equation Method for Regression Analysis VII

- We want to reduce A to the identity matrix by row operations and simultaneously apply these operations to I to produce A^{-1} . To accomplish this we will adjoin the identity matrix to the right side of A , thereby producing a partitioned matrix of the form:

$$[A \mid I]$$

- Then we will apply row operations to this matrix until the left side is reduced to I ; these operations will convert the right side to A^{-1} , so the final matrix will have the form:

$$[I \mid A^{-1}]$$

Normal Equation Method for Regression Analysis VIII

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 2 & 5 & 3 & 0 & 1 & 0 \\ 1 & 0 & 8 & 0 & 0 & 1 \end{array} \right]$$

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & -3 & -2 & 1 & 0 \\ 0 & -2 & 5 & -1 & 0 & 1 \end{array} \right]$$

← We added -2 times the first row to the second and -1 times the first row to the third.

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & -3 & -2 & 1 & 0 \\ 0 & 0 & -1 & -5 & 2 & 1 \end{array} \right]$$

← We added 2 times the second row to the third.

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 3 & 1 & 0 & 0 \\ 0 & 1 & -3 & -2 & 1 & 0 \\ 0 & 0 & 1 & 5 & -2 & -1 \end{array} \right]$$

← We multiplied the third row by -1 .

$$\left[\begin{array}{ccc|ccc} 1 & 2 & 0 & -14 & 6 & 3 \\ 0 & 1 & 0 & 13 & -5 & -3 \\ 0 & 0 & 1 & 5 & -2 & -1 \end{array} \right]$$

← We added 3 times the third row to the second and -3 times the third row to the first.

$$\left[\begin{array}{ccc|ccc} 1 & 0 & 0 & -40 & 16 & 9 \\ 0 & 1 & 0 & 13 & -5 & -3 \\ 0 & 0 & 1 & 5 & -2 & -1 \end{array} \right]$$

← We added -2 times the second row to the first.

$$A^{-1} = \begin{bmatrix} -40 & 16 & 9 \\ 13 & -5 & -3 \\ 5 & -2 & -1 \end{bmatrix} \quad \blacktriangleleft$$

Normal Equation Method vs. Gradient Descent Algorithm

(n is number of features; m is number of examples; α is learning rate)

• Gradient Descent:

- need to select α
- need to do many iterations
- works well with large n
- better results are obtained when applying feature scaling

• Normal Equation:

- don't need to select α
- don't need to iterate – compute in one-step
- slow if n is large (e.g. $n \geq 10^4$)
- need to compute $(X^T X)^{-1}$ is very slow
- if $(X^T X)$ is not invertible – we have problem
- don't need feature scaling

What if $X^T X$ is non-invertible

- some columns (features) are linearly dependent, *i.e.* we have redundant features.

example:

x_1 = house size in feet²

x_2 = house size in meter²

1m = 3.28 feet ← (linearly dependency)

$x_1 = (3.28)^2 x_2$

solution: remove linear dependent features

- too many features ($m \leq n$)
solution: delete some features (e.g. using feature selection algorithm such as mutual information (MI) method or feature compression technique principle component analysis (PCA)), or use regularisation technique because there are too many features for the amount of data that we have

Train, Validation and Test Sets

- **Train:** Which you run your learning algorithm on.
- **Validation/Development(Dev):** Which you use to tune parameters, select features, and make other decisions regarding the learning algorithm.
- **Test:** which you use to evaluate the performance of the algorithm, but not to make any decisions regarding what learning algorithm or parameters to use.

